

고객을 세그멘테이션하자 [프로젝트] - 최홍주

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *
FROM `modulabs_project.data`
LIMIT 10
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	851234	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	844068	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	840296	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	840296	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT *
FROM `modulabs_project.data`

OR

SELECT COUNT(*)
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	851234	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	844068	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	840296	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	840296	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom
11	536367	22745	POPPY'S PLAYHOUSE BEDROOM	6	2010-12-01 08:34:00 UTC	2.1	13047	United Kingdom

- 컬럼 확인하기

컬럼명	의미	설명
InvoiceNo	주문 번호	각각의 고유한 거래를 나타내는 코드.이 코드가 'C'라는 글자로 시작한다면, 취소를 나타냅니다.하나의 거래에 여러 개의 제품이 함께 구매되었다면, 1개의 InvoiceNo에는 여러 개의 StockCode가 연결되어 있습니다.
StockCode	상품 코드	각각의 제품에 할당된 고유 코드
Description	상품 이름 / 설명	각 제품에 대한 설명
Quantity	수량	거래에서 제품을 몇 개 구매했는지에 대한 단위 수
InvoiceDate	주문 날짜 / 시간	거래가 일어난 날짜와 시간
UnitPrice	상품 단가	제품 당 단위 가격(영국 파운드)
CustomerID	고객 ID	각 고객에게 할당된 고유 식별자 코드
Country	국가	주문이 발생한 국가

- 데이터의 스키마 확인하기

스키마	세부정보	미리보기	데이터 탐색기	포러뷰	통계	개보	데이터 프로필	데이터 품질
필터 속성 이름 또는 값 입력								
☐ 필드 이름	유형	모드	설명	키	대조	기본값	정책 태그 ①	데이터 정책
☐ InvoiceNo	STRING	NULLABLE	-	-	-	-	-	-
☐ StockCode	STRING	NULLABLE	-	-	-	-	-	-
☐ Description	STRING	NULLABLE	-	-	-	-	-	-
☐ Quantity	INTEGER	NULLABLE	-	-	-	-	-	-
☐ InvoiceDate	TIMESTAMP	NULLABLE	-	-	-	-	-	-
☐ UnitPrice	FLOAT	NULLABLE	-	-	-	-	-	-
☐ CustomerID	INTEGER	NULLABLE	-	-	-	-	-	-
☐ Country	STRING	NULLABLE	-	-	-	-	-	-

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT
  COUNT(InvoiceNo) AS COUNT_InvoiceNo,
  COUNT(StockCode) AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity) AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice) AS COUNT_UnitPrice,
  COUNT(CustomerID) AS COUNT_CustomerID,
  COUNT(Country) AS COUNT_Country
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

결측치 있는 컬럼 - Description(540455), CustomerID(406829)

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Description	COUNT_Quantity	COUNT_InvoiceDate	COUNT_UnitPrice	COUNT_CustomerID	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
SELECT
  'InvoiceNo' AS column_name,
  ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
  'StockCode' AS column_name,
  ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
  'Description' AS column_name,
  ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL
```

```

SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_per
centage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perc
entage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'Country' AS column_name,
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percent
age
FROM `modulabs_project.data`

```

[결과 이미지를 넣어주세요]

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```

SELECT DISTINCT Description
FROM `modulabs_project.data`
WHERE StockCode = '85123A'

```

[결과 이미지를 넣어주세요]

행	Description
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

결측치 처리

- DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시

```
DELETE
FROM `modulabs_project.data`
WHERE Description IS NULL
OR CustomerID IS NULL
```

[결과 이미지를 넣어주세요]

작업 정보 **결과** 실행 세부정보 실행 그래프

i 이 문으로 data의 행 135,080개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
SELECT *
FROM `modulabs_project.data`
GROUP BY
InvoiceNo,
StockCode,
Description,
Quantity,
InvoiceDate,
UnitPrice,
CustomerID,
Country
HAVING COUNT(*) > 1
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	571034	23239	SET OF 4 KNICK KNACK TINS P...	6	2011-10-13 12:47:00 U
2	571034	23494	VINTAGE DOILY DELUXE SEWIN...	3	2011-10-13 12:47:00 U
3	571034	23245	SET OF 3 REGENCY CAKE TINS	4	2011-10-13 12:47:00 U
4	538826	22749	FELTCRAFT PRINCESS CHARLO...	1	2010-12-14 12:58:00 U
5	577228	22144	CHRISTMAS CRAFT LITTLE FRI...	1	2011-11-18 12:07:00 U
6	577228	23156	SET OF 5 MINI GROCERY MAG...	1	2011-11-18 12:07:00 U
7	577228	23048	SET OF 10 LANTERNS FAIRY LI...	1	2011-11-18 12:07:00 U
8	577228	22770	HAPPY EASTER HANGING DEC...	1	2011-11-18 12:07:00 U

페이지당 결과 수: 50 ▼ 1 - 50 (전체 4837행) |< < > >|

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기

- CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT DISTINCT *
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

작업 정보
결과
실행 세부정보
실행 그래프

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

행	fo_
1	401604

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo 의 개수를 출력하기

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	unique_invoice_c...
1	22190

- 고유한 InvoiceNo 를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
FROM `modulabs_project.data`
LIMIT 100
```

[결과 이미지를 넣어주세요]

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318

- InvoiceNo 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74219	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
6	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
7	C547388	22784	LANTERN CREAM GAZERBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
8	C547388	37445	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
9	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
10	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)/ COUNT(*) * 100, 1) AS canceled_percentage
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	canceled_percent...
1	2.2

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode_count
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	unique_stockcod...
1	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10
```

[결과 이미지를 넣어주세요]

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- StockCode** 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고

행	number_count	stock_cnt
1	5	3676
2	0	7
3	1	1

- 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
    SELECT StockCode,
        LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `modulabs_project.data`
)
WHERE number_count IN (0,1)
```

[결과 이미지를 넣어주세요]

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- StockCode의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```
SELECT ROUND(COUNT(*) / (SELECT COUNT(*) FROM `modulabs_project.data`) * 100, 2) AS percentage
FROM (
    SELECT StockCode,
        LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `modulabs_project.data`
)
WHERE number_count IN (0, 1)
```

```
# 몇 개 없어서 이렇게 하는 방법도 있음 위에 코드랑 같은 결과 나옴
SELECT ROUND(SUM(CASE WHEN StockCode IN ('POST','M','C2','D','BANK CHARGES','PADS','DOT','CRUK')
    THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS percentage
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	percentage
1	0.48

- 제품과 관련되지 않은 거래 기록을 제거하기

```
DELETE
FROM `modulabs_project.data`
WHERE StockCode IN (
    SELECT DISTINCT StockCode
    FROM (
        SELECT StockCode,
            LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
        FROM `modulabs_project.data`
    )
    WHERE number_count IN (0,1)
)
```

[결과 이미지를 넣어주세요]

i 이 문으로 data의 행 1,915개가 삭제되었습니다.

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT DISTINCT Description, COUNT(*) AS description_cnt
FROM `modulabs_project.data`
GROUP BY Description
LIMIT 30
```

[결과 이미지를 넣어주세요]

행	Description	description_cnt
1	MEDIUM CERAMIC TOP STORA...	208
2	AIRLINE BAG VINTAGE JET SET...	96
3	FOUR HOOK WHITE LOVEBIRDS	265
4	BATHROOM METAL SIGN	60
5	CLEAR DRAWER KNOB ACRYLI...	331
6	RED TOADSTOOL LED NIGHT LI...	539
7	SET OF 2 TINS VINTAGE BATHR...	59
8	BLACK EAR MUFF HEADPHONES	18
9	GREEN DRAWER KNOB ACRYLI...	149
10	BLACK GRAND BAROQUE PHOT...	7

- 소문자가 섞여있는 행

행	Description
1	BAG 250g SWIRLY MARBLES
2	3 TRADITIONAL BISCUIT CUTTE...
3	BAG 125g SWIRLY MARBLES
4	POLYESTER FILLER PAD 30CMx...
5	BAG 500g SWIRLY MARBLES
6	POLYESTER FILLER PAD 45x45...
7	POLYESTER FILLER PAD 40x40...
8	ESSENTIAL BALM 3.5g TIN IN E...
9	High Resolution Image
10	FRENCH BLUE METAL DOOR SI...

- 서비스 관련 정보를 포함하는 행들을 제거하기

```
DELETE
FROM `modulabs_project.data`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image')
```

[결과 이미지를 넣어주세요]

작업 정보	결과	실행 세부정보	실행 그래프
i 이 문으로 data의 행 83개가 삭제되었습니다.			

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

작업 정보	결과	실행 세부정보	실행 그래프
이 문으로 이름이 data인 테이블이 교체되었습니다.			

UnitPrice 살펴보기

- UnitPrice의 최솟값, 최댓값, 평균을 구하기

```
SELECT MIN(UnitPrice) AS min_price, MAX(UnitPrice) AS max_price, AVG(UnitPrice) AS avg_price
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757406...

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
SELECT COUNT(Quantity) AS cnt_quantity, MIN(Quantity) AS min_quantity, MAX(Quantity) AS max_quantity,
AVG(Quantity) AS avg_quantity
FROM `modulabs_project.data`
WHERE UnitPrice = 0
```

[결과 이미지를 넣어주세요]

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- UnitPrice = 0를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT *
FROM `modulabs_project.data`
WHERE UnitPrice != 0
```

[결과 이미지를 넣어주세요]

작업 정보	결과	실행 세부정보	실행 그래프
이 문으로 이름이 data인 테이블이 교체되었습니다.			

11-7. RFM 스코어

Recency

- InvoiceDate 컬럼을 연월일 자료형으로 변경하기

```
SELECT DATE(InvoiceDate) AS InvoiceDay,*
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	
3	2010-12-07	537626	22195	12	2010-12-07 14:57:00 UTC	1.65	
4	2010-12-07	537626	84969	6	2010-12-07 14:57:00 UTC	4.25	
5	2010-12-07	537626	85232D	3	2010-12-07 14:57:00 UTC	4.95	
6	2010-12-07	537626	22492	36	2010-12-07 14:57:00 UTC	0.65	
7	2010-12-07	537626	21171	12	2010-12-07 14:57:00 UTC	1.45	
8	2010-12-07	537626	22375	4	2010-12-07 14:57:00 UTC	4.25	
9	2010-12-07	537626	22726	4	2010-12-07 14:57:00 UTC	3.75	
10	2010-12-07	537626	22728	4	2010-12-07 14:57:00 UTC	3.75	

가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT MAX(InvoiceDate) AS most_recent_date
FROM `modulabs_project.data`
```

[결과 이미지를 넣어주세요]

행	most_recent_date
1	2011-12-09

유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
SELECT
  CustomerID,
  MAX(InvoiceDate) AS InvoiceDay
FROM `modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17

가장 최근 일자(most_recent_date)와 유저별 마지막 구매일(InvoiceDay)간의 차이를 계산하기

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDate) OVER () - InvoiceDate) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(InvoiceDate) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

행	CustomerID	recency
1	12427	11
2	12430	43
3	12653	148
4	12657	11
5	12662	0
6	12740	63
7	12763	138
8	12928	35
9	13531	205
10	13565	19

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```

CREATE OR REPLACE TABLE `modulabs_project.user_r` AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `modulabs_project.data`
  GROUP BY CustomerID
)

```

[결과 이미지를 넣어주세요]

행	CustomerID	recency
1	18102	0
2	14422	0
3	16954	0
4	13777	0
5	15694	0
6	17001	0
7	16446	0
8	14051	0
9	14446	0
10	17428	0

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```

SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `modulabs_project.data`
GROUP BY CustomerID

```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1
10	12356	3

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_rf` AS
```

```
-- (1) 전체 거래 건수 계산
```

```
WITH purchase_cnt AS (
```

```
  SELECT
```

```
    CustomerID,
```

```
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
```

```
  FROM `modulabs_project.data`
```

```
  GROUP BY CustomerID
```

```
),
```

```
-- (2) 구매한 아이템 총 수량 계산
```

```
item_cnt AS (
```

```
  SELECT
```

```
    CustomerID,
```

```
    SUM(Quantity) AS item_cnt
```

```
  FROM `modulabs_project.data`
```

```
  GROUP BY CustomerID
```

```
)
```

```
-- 기존의 user_r에 (1)과 (2)를 통합
```

```
SELECT
```

```
  pc.CustomerID,
```

```
  pc.purchase_cnt,
```

```
  ic.item_cnt,
```

```
  ur.recency
```

```
FROM purchase_cnt AS pc
```

```
JOIN item_cnt AS ic
```

```
  ON pc.CustomerID = ic.CustomerID
```

```
JOIN `modulabs_project.user_r` AS ur
```

```
  ON pc.CustomerID = ur.CustomerID;
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	15520	1	314	1
3	13436	1	76	1
4	13298	1	96	1
5	14569	1	79	1
6	14204	1	72	2
7	15195	1	1404	2
8	15471	1	256	2
9	17914	1	457	3
10	16569	1	93	3

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```
SELECT
  CustomerID,
  ROUND(SUM(UnitPrice * Quantity), 1) AS user_total
FROM `modulabs_project.data`
GROUP BY CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.6
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

CustomerID = 12346 토탈 금액이 0이 나와서 확인해봄

구매 후 16분 뒤 취소한걸로 보임 그 전후로 구매한 적은 없어서 토탈 금액이 0인걸로 확인!

행	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description	line_total
1	541431	22166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...	77183.6
2	541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...	-77183.6

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) data 테이블을 user_rf 테이블과 조인(LEFT JOIN) 한 후, 2) purchase_cnt 로 나누어서 3) user_rfm 테이블로 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ut.user_total / rf.purchase_cnt AS user_average
FROM `modulabs_project.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    ROUND(SUM(UnitPrice * Quantity), 1) AS user_total
```

```
FROM `modulabs_project.data`
GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13298	1	96	1	360.0	360.0
3	15520	1	314	1	343.5	343.5
4	14569	1	79	1	227.4	227.4
5	13436	1	76	1	196.9	196.9
6	15471	1	256	2	454.5	454.5
7	14204	1	72	2	150.6	150.6
8	15195	1	1404	2	3861.0	3861.0
9	17914	1	457	3	329.4	329.4
10	12478	1	233	3	546.0	546.0

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```
SELECT *
FROM `modulabs_project.user_rfm`
```

[결과 이미지를 넣어주세요]

작업 정보

결과

시각화

JSON

실행 세부정보

실행 그래프

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13298	1	96	1	360.0	360.0
3	15520	1	314	1	343.5	343.5
4	14569	1	79	1	227.4	227.4
5	13436	1	76	1	196.9	196.9
6	15471	1	256	2	454.5	454.5
7	14204	1	72	2	150.6	150.6
8	15195	1	1404	2	3861.0	3861.0
9	17914	1	457	3	329.4	329.4
10	12478	1	233	3	546.0	546.0

페이지당 결과 수: 501 - 50

전체 4362행

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) user_rfm 테이블과 결과를 합치기
- 3) user_data 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `modulabs_project.data`
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	16061	1	-1	269	-29.9	-29.9	1
2	15668	1	72	217	76.3	76.3	1
3	17102	1	2	261	25.5	25.5	1
4	15657	1	24	22	30.0	30.0	1
5	12943	1	-1	301	-3.8	-3.8	1
6	14576	1	12	372	35.4	35.4	1
7	16737	1	288	53	417.6	417.6	1
8	13302	1	5	155	63.8	63.8	1
9	16738	1	3	297	3.8	3.8	1
10	17956	1	1	249	12.8	12.8	1

```
SELECT *
FROM `modulabs_project.user_data`
ORDER BY unique_products DESC
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	14911	242	76823	1	128768.2	532.1	1791
2	12748	217	23516	0	29820.0	137.4193548387...	1767
3	17841	169	22613	1	39861.5	235.8668639053...	1330
4	14096	17	16336	4	53258.4	3132.847058823...	1118
5	14298	45	58021	3	50862.4	1130.27555555...	884
6	14606	125	5932	1	11486.6	91.8928000000...	829
7	14769	10	7208	2	10382.2	1038.22	717
8	14156	64	56896	9	113685.8	1776.340625	714
9	14646	73	196556	1	278778.0	3818.876712328...	699
10	13089	118	30742	2	57322.1	485.7805084745...	636

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 평균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합

```
CREATE OR REPLACE TABLE `modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_in
  terval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DA
    Y) AS interval_
    FROM
      `modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM `modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	13068	2	200	10	344.0	172.0	1	309.0
2	18084	2	311	16	87.5	43.75	2	284.0
3	18080	2	642	18	1231.5	615.75	2	223.0
4	12875	2	2019	143	343.2	171.6	1	219.0
5	14777	2	-9	4	-17.4	-8.7	2	176.5
6	18273	3	80	2	204.0	68.0	1	127.5
7	17029	2	400	110	716.0	358.0	1	126.0
8	14865	2	108	7	52.2	26.1	4	121.33
9	14987	2	128	15	326.4	163.2	2	116.33
10	18017	3	300	81	523.0	174.333333333...	1	114.5

페이지당 결과 수: 50 1 ~ 50 (전체 4362행) |< >

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기
 - 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
 - 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
 - 취소 빈도와 취소 비율을 계산하고 그 결과를 **user_data**에 통합하기
(취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE `modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `modulabs_project.data`
  GROUP BY CustomerID
)

SELECT u.*, t.* EXCEPT(CustomerID), ROUND(t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM `modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	16148	1	72	296	76.3	76.3	1	0.0
2	16565	1	46	364	173.7	173.7	3	0.0
3	13739	1	54	18	216.9	216.9	3	0.0
4	13645	1	158	120	252.1	252.1	5	0.0
5	14039	1	92	33	152.2	152.2	6	0.0
6	16213	1	48	78	159.0	159.0	6	0.0
7	14890	1	51	253	125.7	125.7	7	0.0
8	17022	1	108	31	71.0	71.0	7	0.0
9	14143	1	30	133	115.8	115.8	7	0.0
10	15986	1	268	30	168.1	168.1	14	0.0

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 **user_data**를 출력하기

```
SELECT *
FROM `modulabs_project.user_data`
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	16148	1	72	296	76.3	76.3	1	0.0	1	0	0.0
2	16565	1	46	364	173.7	173.7	3	0.0	1	0	0.0
3	13739	1	54	18	216.9	216.9	3	0.0	1	0	0.0
4	13645	1	158	120	252.1	252.1	5	0.0	1	0	0.0
5	14039	1	92	33	152.2	152.2	6	0.0	1	0	0.0
6	16213	1	48	78	159.0	159.0	6	0.0	1	0	0.0
7	14890	1	51	253	125.7	125.7	7	0.0	1	0	0.0
8	17022	1	108	31	71.0	71.0	7	0.0	1	0	0.0
9	14143	1	30	133	115.8	115.8	7	0.0	1	0	0.0
10	15986	1	268	30	168.1	168.1	14	0.0	1	0	0.0

페이지당 결과 수: 50 1 ~ 50 (전체 4362행) |< >

회고

[회고 내용을 작성해주세요]

Keep : 이 프로젝트를 통해 그동안 배웠던 SQL을 총정리하면서 복습하는 시간이 되어 좋았습니다.

Problem : 중간중간 배웠던 것 같은데 기억이 나지 않는 함수들과 모르는 함수들도 많아서 열심히 리서치하면서 진행한 것 같습니다.

Try : 짧은 시간에 많은 개념을 진행하다 보니 아직 익숙하지 않은 부분이 많은 것 같아서 역시 복습하는 시간을 더 늘려야 할 것 같습니다.